
BE : DÉTERMINATION DU MOUVEMENT APPARENT SUR DES IMAGES

Consignes générales :

- Par groupe de 5 , 6 étudiants maximum, vous devez rédiger un rapport dans lequel vous répondrez aux questions du sujet et plus globalement au problème demandé. La qualité de rédaction ainsi que tous les compléments que vous apporterez (réflexions, analyses des réponses, tests sur d'autres données, etc...) seront fortement pris en compte. Ce rapport devra être rédigé sous un traitement de texte (les pages manuscrites ne sont pas acceptées) et rendu sous la forme d'un fichier .pdf.
 - Toutes les fonctions programmées le seront en Python et seront abondamment commentées. Le rendu du projet se fera sous la forme d'un fichier .zip contenant le rapport ainsi que l'ensemble des programmes et données nécessaires à l'exécution des programmes.
 - Les fonctions programmées doivent l'être dans un fichier librairie ayant pour nom : **Ma223_BE_lib.py**. Dans ce fichier les différentes fonctions doivent être regroupées dans des blocs (commande : #%%) correspondants aux parties du sujet.
 - Les applications numériques des fonctions du sujet seront dans un fichier ayant pour nom : **Ma223_BE_main.py**. Dans ce fichier les différents cas d'études doivent être regroupés dans des blocs séparés.
 - Les différentes images devront être enregistrées dans un dossier ayant pour nom **Images**. Ce dossier doit être mis dans le même répertoire que vos fichiers Python **Ma223_BE_lib.py** et **Ma223_BE_main.py**.
 - Les noms des différents membres du groupe doivent apparaître en commentaire en début de fichier.
 - Le non respect des consignes précédentes entraînera des points de pénalités pouvant être conséquents...
 - Les questions d'analyses de résultats sont généralement ouvertes. La qualité de vos réflexions sera fortement prise en compte pour la notation de ces questions.
 - **Il est impératif de lire le sujet en entier.**
 - **Enfin on rappelle qu'un BE est un examen et comme tout examen il arrive de ne pas pouvoir répondre à toutes les questions.**
-

Avant de débiter ce sujet on rappelle que vous trouverez dans la première partie du fascicule de cours de Ma223 quelques notions de Python utiles pour ce BE. En particulier vous trouverez comment installer la librairie OpenCV et quelques fonctions de base de cette librairie. On rappelle simplement que l'on importe une image en nuances de gris grâce à la librairie OpenCV en utilisant les commandes :

```
import cv2
image=cv2.imread('frame13.png',0)
```

Une image « standard » i.e. couleur est composée de trois matrices, chacune représentant les intensités des couleurs primaires BGR (bleu, vert, rouge) lorsque l'on utilise la fonction imread d'OpenCV et RGB (rouge, vert, bleu) avec PIL et Matplotlib. Vous pouvez voir dans l'explorateur de variable qu'une image est en réalité la donnée de trois matrices : une matrice pour l'intensité du rouge, une pour le vert et une couche pour le bleu. Chacune de ces intensités est codée par un « entier » (type uint8) compris entre 0 et 255 (i.e. sur 2^8 bits). Les images en nuances de gris possèdent une unique couche et sont donc des matrices. **Attention les images étant des array de type uint8 il faudra absolument penser à convertir vos images en matrice de flottants lorsque vous voulez réaliser des opérations mathématiques dessus (sous entendu avec des réels) et réciproquement les reconverter en uint8 lorsque vous voulez les afficher.** Par exemple que tester ce que renvoie la commande suivante :

```
a=np.array([255,-256,-1]).astype('uint8')
```

Observer la différence entre les deux scripts suivants :

— Version 'uint8' :

```
a=np.array([0,255]).astype('uint8')
print(a+a)
```

— Version 'float' :

```
a=np.array([0,255])
print(a+a)
```

Ce projet se compose de 4 parties. La première représente une étape de formalisation du problème de l'appréhension du mouvement dans une séquence d'images (=vidéo). Elle sous-tend à toutes les autres parties du sujet. En particulier nous verrons que dans sa plus grande généralité ce problème n'admet pas de solution bien définie. On appelle ce problème : **le problème de l'ouverture**. En effet une image est la projection bidimensionnelle d'une scène tridimensionnelle, ce qui ne permet pas d'avoir accès à la « profondeur ».

La seconde et la troisième partie de ce projet permettent néanmoins de résoudre ce problème en admettant des hypothèses réductrices mais nécessaires à la résolution. Ces deux parties peuvent être traitées séparément mais elles utilisent chacune la partie 1.

Enfin dans une dernière partie on comparera ces deux méthodes à travers des applications de celles-ci.

Attention certains calculs peuvent être longs (il est conseillé de ne pas dépasser une centaine d'itérations pour la partie 3 par exemple). Afin de rendre votre programme plus rapide il est possible d'utiliser la librairie Numba dont vous trouverez une rapide introduction sur le lien : <https://numba.pydata.org/numba-doc/dev/user/5minguide.html>.

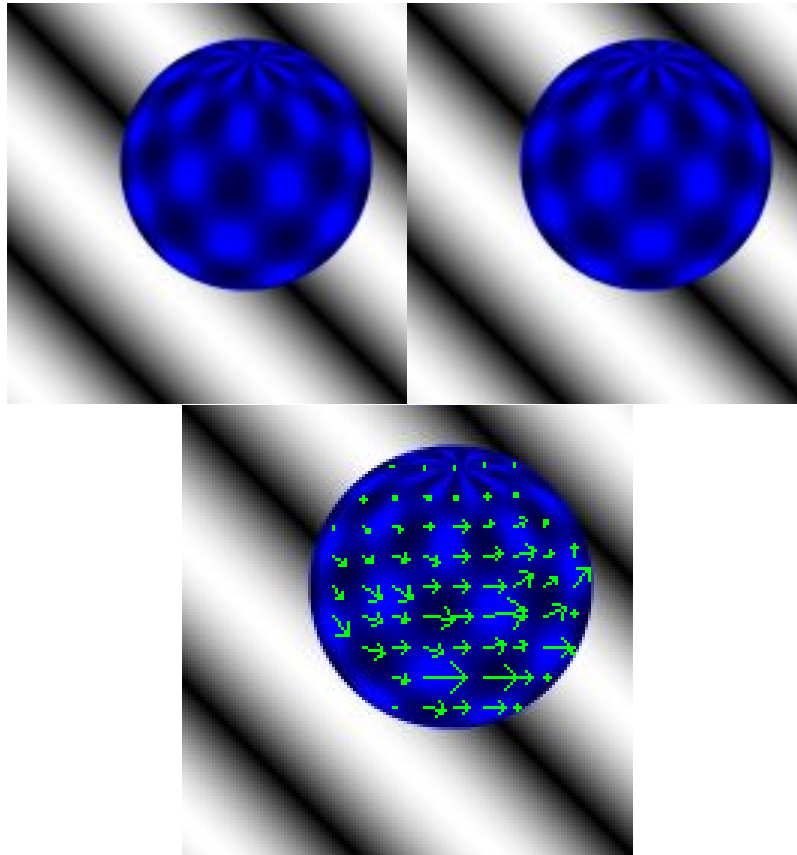


FIGURE 1 – Dans ce BE, nous allons à partir de deux images successives d'une scène, les images du haut, de gauche à droite pour l'ordre, essayer de retrouver le « mouvement » des objets dans cette scène, image en bas (estimation du « flux optique »). On voit que l'on retrouve le fait que la sphère pivote entre l'image 1 (en haut à gauche) et l'image 2 (en haut à droite). Cette rotation est représentée par une estimation des vecteurs vitesses (flèches vertes) de (quelques) points sur la sphère dans l'image du bas.



FIGURE 2 – Un exemple de segmentation d'images à partir du flux optique généré par 2 images successives d'une même scène (image 1 et 2 en partant de la gauche). Les deux dernières images sont le résultat de la segmentation. Ici on a détecté les véhicules en mouvement.

Première partie

Détermination du flux optique : formalisation du problème

Dans ce BE nous allons nous intéresser au problème de la détermination du mouvement à partir des images d'une vidéo. Une vidéo est en général la restitution planaire i.e. 2D d'une scène 3D et le mouvement réel en trois dimensions n'est en pratique pas déterminable à partir de ces données 2D uniquement. Ce que l'on perçoit dans une vidéo est le mouvement apparent, ce que l'on nomme **le flux optique**. Dans ce BE on pourra utiliser les données tests fournis par le site du Middlebury College et son équipe recherche en vision assistée par ordinateur. On trouvera cette base de donnée sur Moodle ainsi que sur la page : <https://vision.middlebury.edu/flow/data/>.

Pour simplifier on considérera des images en noir et blanc afin d'avoir des matrices (et non des triplets de matrices pour les images en couleurs). Formellement une vidéo est la donnée d'une suite de matrices $\left(F^{(t)}\right)_{t \in \llbracket 0, q \rrbracket}$ où $q \in \mathbb{N}^*$ représente le nombre d'images ($q + 1$ ici) et où pour tout $t \in \llbracket 0, q \rrbracket$, $F^{(t)} \in \mathcal{M}_{m,n}(\mathbb{R})$ avec m, n deux entiers strictement positifs, représente une image.

Afin d'utiliser les outils du calcul différentiel on supposera qu'il existe une fonction à trois variables :

$$f : [0, t_f] \times [0, 1]^2 \rightarrow \mathbb{R}$$

avec $t_f \in \mathbb{R}_+^*$, que l'on supposera de classe $\mathcal{C}^\infty$ et qui lorsqu'on discrétise l'intervalle de temps et le carré unité d'espace on retrouve la suite de matrice $\left(F^{(t)}\right)_{t \in \llbracket 1, q \rrbracket}$ formant la vidéo i.e. en posant :

$$\begin{cases} t_r := r \Delta t & \forall r \in \llbracket 0, q \rrbracket & \Delta t := \frac{t_f}{q} \\ x_i := (i - 1) \Delta x & \forall i \in \llbracket 1, m \rrbracket & \Delta x := \frac{1}{m - 1} \\ y_j := (j - 1) \Delta y & \forall j \in \llbracket 1, n \rrbracket & \Delta y := \frac{1}{n - 1} \end{cases}$$

tels que l'on ait la relation :

$$f(t_r, x_i, y_j) = F_{i,j}^{(r)}$$

Or ce que nous cherchons à déterminer c'est le mouvement apparent d'un pixel/point fixé sur une image tout au long de la vidéo i.e. une courbe paramétrique de la forme :

$$\begin{cases} c : [0, t_f] \rightarrow [0, 1]^2 \\ c(0) = (x_0, y_0) \in [0, 1]^2 \end{cases}$$

où l'on notera sous la forme d'un vecteur colonne :

$$c(t) := (x(t), y(t))^T$$

En particulier le **flux optique** associé à un point (x_0, y_0) au temps t est défini comme étant le vecteur vitesse :

$$c'(t) := (x'(t), y'(t))^T$$

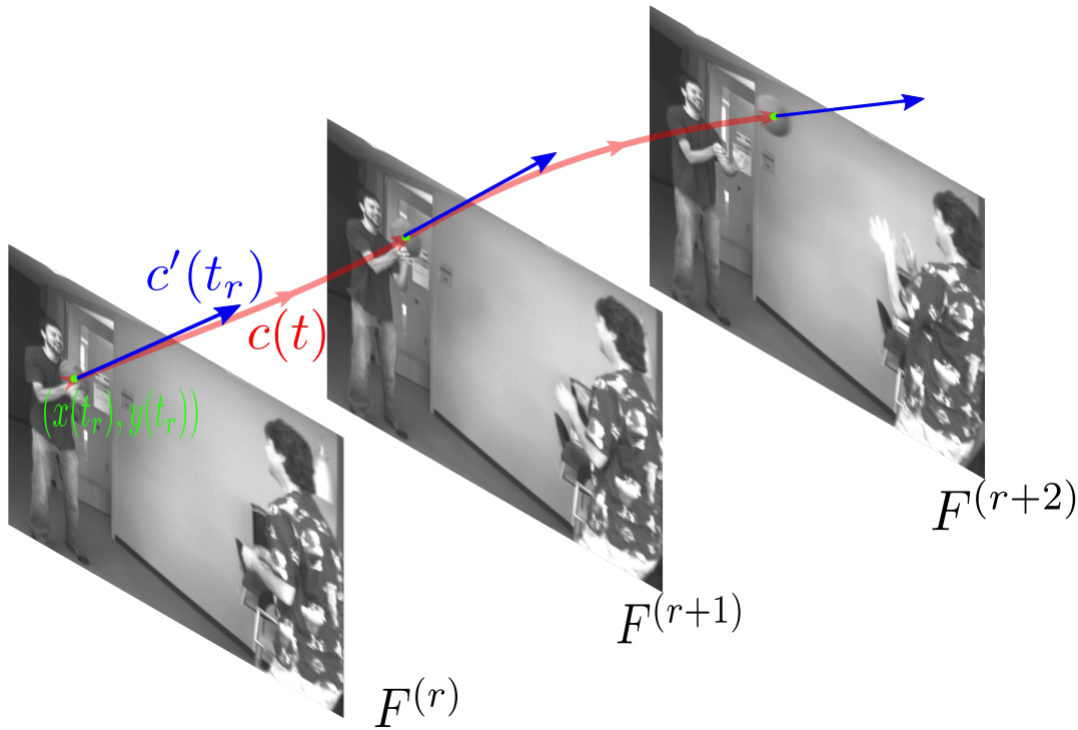


FIGURE 3 – Représentation schématique du mouvement apparent d'un point de l'image (courbe rouge c) et du flux optique associé (courbe bleue c'). C'est la détermination de cette courbe pour chaque point de l'image qui va être l'objectif de ce BE.

On supposera que la couleur d'un point suivi ne change pas i.e. que l'on a pour tout $t \in [0, t_f]$:

$$f(t, x(t), y(t)) = \alpha$$

pour un α réel fixé. Cette hypothèse est « raisonnable » pour des « petits » déplacements et lorsqu'il n'y a pas de changement de luminosité brusque.

1. Montrer que la condition précédente implique l'équation aux dérivées partielles suivantes :

$$\boxed{\frac{\partial f}{\partial t}(t, c(t)) + \nabla_{x,y} f(t, c(t))^T c'(t) = 0} \quad (1)$$

où $\nabla_{x,y} f$ note le gradient « spatial » i.e. le gradient en x et y seulement : $\nabla_{x,y} f := \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)^T$.

2. Fixons un $t \in [0, t_f]$ et notons $c'(t) := (v_1, v_2)^T \in \mathbb{R}^2$. L'équation (1) s'écrit alors :

$$\frac{\partial f}{\partial t} = -\nabla_{x,y} f^T c_t$$

Justifier que même si $\frac{\partial f}{\partial t}$ et $\nabla_{x,y} f$ sont connues, c_t n'est pas déterminée de manière unique.

3. Retournons à notre version originelle discrète du problème du calcul du flux optique. Supposons fixées deux images $F^{(r)}$ et $F^{(r+1)}$. La valeur de $f(t_r, x(t_r), y(t_r))$ correspond alors au coefficient :

$$F_{i,j}^{(r)}$$

pour un i et un j dans $\llbracket 1, m \rrbracket$ et $\llbracket 1, n \rrbracket$ respectivement. Avec ces notations, on cherche donc à déterminer le vecteur : $c'(t_r)$ en ce point $(x(t_r), y(t_r)) := (x_i, y_j)$. Comme pour les matrices un « pas » ne peut pas être plus petit que 1 (=l'écart entre deux pixels) on imposera dans la suite :

$$\Delta x = \Delta y = \Delta t = 1$$

en « identifiant » les intervalles continus et discrets :

$$\begin{cases} [0, 1] \stackrel{\phi_1}{\simeq} \llbracket 1, m \rrbracket \\ [0, 1] \stackrel{\phi_2}{\simeq} \llbracket 1, n \rrbracket \\ [0, t_f] \stackrel{\phi_3}{\simeq} \llbracket 0, q \rrbracket \end{cases}$$

où les ϕ_i sont les transformations affines croissantes de changement d'intervalles. Ainsi avec cette identification un point (x_i, y_j) sera représenté/identifié au couple (i, j) .

(a) Justifier que l'on peut prendre comme approximation les formules suivantes :

$$\begin{cases} \frac{\partial f}{\partial x}(t_r, x(t_r), y(t_r)) \simeq \frac{1}{4} (F_{i+1,j}^{(r+1)} - F_{i-1,j}^{(r+1)} + F_{i+1,j}^{(r)} - F_{i-1,j}^{(r)}) \\ \frac{\partial f}{\partial y}(t_r, x(t_r), y(t_r)) \simeq \frac{1}{4} (F_{i,j+1}^{(r+1)} - F_{i,j-1}^{(r+1)} + F_{i,j+1}^{(r)} - F_{i,j-1}^{(r)}) \\ \frac{\partial f}{\partial t}(t_r, x(t_r), y(t_r)) \simeq F_{i,j}^{(r+1)} - F_{i,j}^{(r)} \end{cases}$$

On notera de manière plus concise les approximations obtenues par les formules précédentes :

$$\begin{cases} \partial_x F_{i,j}^{(r)} \simeq \frac{\partial f}{\partial x}(t_r, x(t_r), y(t_r)) \\ \partial_y F_{i,j}^{(r)} \simeq \frac{\partial f}{\partial y}(t_r, x(t_r), y(t_r)) \\ \partial_t F_{i,j}^{(r)} \simeq \frac{\partial f}{\partial t}(t_r, x(t_r), y(t_r)) \end{cases}$$

(b) Rédiger une fonction **approx_f** sous Python qui prend en entrée deux matrices (=les deux images en noir et blanc) de taille m, n , deux entiers i et j dans $\llbracket 1, m \rrbracket$ et $\llbracket 1, n \rrbracket$ respectivement et qui renvoie les trois approximations précédentes : $\partial_x F_{i,j}^{(r)}, \partial_y F_{i,j}^{(r)}, \partial_t F_{i,j}^{(r)}$. On imposera comme conditions aux bords :

$$\begin{cases} F_{i,0}^{(r)} := F_{i,1}^{(r)} & F_{0,j}^{(r)} := F_{1,j}^{(r)} \\ F_{i,n+1}^{(r)} := F_{i,n}^{(r)} & F_{m+1,j}^{(r)} := F_{m,j}^{(r)} \end{cases}$$

pour tout $(i, j) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket$. **Plus généralement on imposera des conditions aux bords constantes lorsque ces uniques conditions ne suffisent pas (par exemple si l'on a besoin de $F_{i,-1}^{(r)}$ dans un calcul numérique).**

(c) En utilisant les images tests fournies sur Moodle et la fonction `plt.imshow` de Matplotlib afficher les trois matrices $\partial_x F^{(r)}, \partial_y F^{(r)}$ et $\partial_t F^{(r)}$ pour certains couples d'images tests. Commenter les résultats.

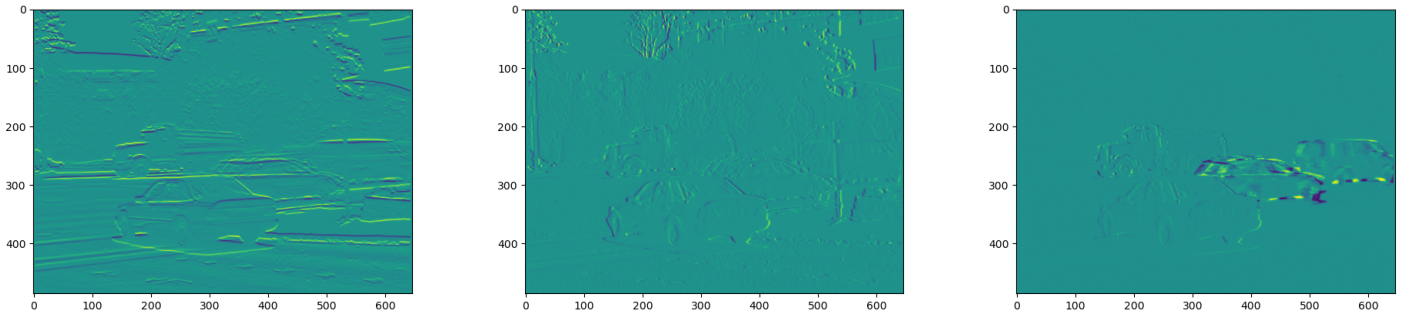


FIGURE 4 – Un exemple d'affichage des matrices $\partial_x F^{(r)}, \partial_y F^{(r)}$ et $\partial_t F^{(r)}$ de gauche à droite respectivement. Cet exemple a été réalisé avec les frames 10 et 11 de Dumptruck.

Deuxième partie

En utilisant les moindres carrés.

L'impossibilité de déterminer de manière unique le flux optique traduit que le mouvement perçu sur les images (=flux optique) est une projection d'un mouvement tridimensionnel et comme toute projection qui se respecte, elle engendre une perte « d'information ». **Dans cette section nous allons essayer de combler ces informations manquantes en faisant l'hypothèse « simplificatrice » qu'un point fixé sur l'image et les points voisins de celui-ci ont même flux optique localement.**

Supposons donc fixé un pixel/point sur une image i.e. on le rappelle un $F_{i,j}^{(r)}$ avec $(i, j, r) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket \times \llbracket 0, q-1 \rrbracket$ et notons :

$$\begin{cases} v_1 := c'(t_r)_1 \\ v_2 := c'(t_r)_2 \end{cases}$$

Avec ces notations et les notations et conventions de la partie 1 l'approximation de l'équation (1) s'écrit de la manière suivante :

$$\partial_t F_{i,j}^{(r)} + \partial_x F_{i,j}^{(r)} v_1 + \partial_y F_{i,j}^{(r)} v_2 = 0$$

Fixons un entier $h \in \mathbb{N}^*$ et notons V_h le voisinage de taille $2h$ du point $F_{i,j}^{(r)}$ i.e. le tableau :

$$V_h := \begin{pmatrix} (x_{i-h}, y_{j-h}) & \cdots & \cdots & \cdots & (x_{i-h}, y_{j+h}) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \vdots & \cdots & (x_i, y_j) & \cdots & \vdots \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ (x_{i+h}, y_{j-h}) & \cdots & \cdots & \cdots & (x_{i+h}, y_{j+h}) \end{pmatrix}$$

L'hypothèse simplificatrice s'écrit alors sous la forme d'un système rectangulaire :

$$(\mathcal{S}) := \begin{cases} \partial_x F_{i-h,j-h}^{(r)} v_1 + \partial_y F_{i-h,j-h}^{(r)} v_2 = -\partial_t F_{i-h,j-h}^{(r)} \\ \vdots \\ \partial_x F_{i,j}^{(r)} v_1 + \partial_y F_{i,j}^{(r)} v_2 = -\partial_t F_{i,j}^{(r)} \\ \vdots \\ \partial_x F_{i+h,j+h}^{(r)} v_1 + \partial_y F_{i+h,j+h}^{(r)} v_2 = -\partial_t F_{i+h,j+h}^{(r)} \end{cases}$$

1. Pouvez-vous critiquer l'hypothèse simplificatrice précédente ? Quels cas peut-on imaginer qui ne respecteraient pas cette hypothèse ?
2. Justifier que le système $(\mathcal{S})$ s'écrit sous la forme matricielle suivante :

$$A^{(i,j)} v^{(i,j)} = b^{(i,j)} \quad (\mathcal{S}_M)$$

où $A^{(i,j)} \in \mathcal{M}_{(2h+1)^2, 2}(\mathbb{R})$, $A^{(i,j)} \in \mathbb{R}^{(2h+1)^2}$ et $v^{(i,j)} := (v_1, v_2)^T$ est le vecteur inconnu à déterminer. On pourra choisir comme ordre de « numérotation » celui pris par Numpy lorsqu'on applique la fonction **M.reshape(-1,1)** à une matrice/array M. Lorsqu'il n'y aura pas d'ambiguïté sur le point (i, j) choisi on omettra les indices (i, j) supérieurs afin d'alléger les notations.

3. Rappeler la condition sur le rang de A pour que le système matriciel $(\mathcal{S}_M)$ admette une unique solution au sens des moindres carrés. Montrer que les valeurs propres de $A^T A$ sont toutes supérieures ou égales à 0. Ainsi un indicateur de « confiance » sur la qualité du calcul effectué sera de regarder :

$$\lambda_{i,j} = \min_{\lambda \in \text{Spec}(A^T A)} \lambda$$

Pouvez-vous imaginer des configurations de pixels sur les deux images qui impliqueraient que le rang de A soit strictement inférieur à 2 ?

4. Créer une fonction **MC_flux_calc** qui prendra en entrée deux images consécutives d'une vidéo $F^{(r)}$ et $F^{(r+1)}$ et qui renverra deux matrices V_1 et V_2 approximations du flux optique ainsi qu'une matrice Λ des minima des valeurs propres de chacun des problèmes de moindres carrés résolus. Ces deux matrices seront calculées en utilisant l'algorithme suivant :

Algorithme 1 Résumer de l'algorithme de calcul du flux optique à l'aide des moindres carrés

Fixer un entier $h \in \mathbb{N}^*$.

Calculer les $\partial_x F_{i,j}^{(r)}, \partial_y F_{i,j}^{(r)}$ et $\partial_t F_{i,j}^{(r)}$ pour tout $(i, j) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket$.

Initialiser les matrices de taille (m, n) des composantes du flux optique :

$$V_1 = 0_{m,n} \text{ et } V_2 = 0_{m,n}$$

Initialiser une matrice Λ nulle de taille (m, n) .

Pour tout $(i, j) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket$:

Si $rg(A^{(i,j)}) = 2$:

Résoudre au sens des moindres carrés :

$$A^{(i,j)} v^{(i,j)} = b^{(i,j)}$$

Stocker : $V_{1,i,j} := v_1^{(i,j)}, V_{2,i,j} := v_2^{(i,j)}$ et $\Lambda_{i,j} = \min_{\lambda \in \text{Spec}(A^{(i,j)T} A^{(i,j)})} \lambda$

Sinon :

Stocker : $V_{1,i,j} := 0, V_{2,i,j} := 0$ et $\Lambda_{i,j} = 0$

Vous pourrez tester votre algorithme sur les données fournies sur Moodle et la résolution des moindres carrés pourra être effectuée avec l'algorithme de Cholesky modifié si vous utilisez Numba (sinon `linalg.solve` si cela est trop long...). De plus il est possible d'afficher le résultat obtenu en vous inspirant du code suivant (on rappelle que dans OpenCV les conventions lignes et colonnes sont inversées par rapport aux conventions mathématiques) :

```
image=np.copy(img1)
image=cv2.cvtColor(image, cv2.COLOR_GRAY2RGB)
for i in range(0,m,10):
    for j in range(0,n,10):
        u,v=int(V1[i,j]), int(V2[i,j])
        if np.sqrt(u**2+v**2)>1:
            image = cv2.arrowedLine(image, (j,i), (j+v,i+u), (0,255,0),1, tipLength = 0.5)
cv2.imshow("affichage_final", image)
```

où `img1` représente la première des deux images considérées. On affichera également la matrice Λ en utilisant la commande `plt.imshow` de Matplotlib afin de contrôler la « qualité » de notre évaluation du flux optique.

- En utilisant la librairie OpenCV et `cv2.VideoWriter` (cf. fascicule de cours), réaliser une sortie vidéo des calculs du flux optique sur une série d'images. On pourra accoler aux images du flux optique les images correspondant aux matrices Λ . Si les images fournies en exemple ne vous suffisent pas vous pouvez tester votre algorithme sur n'importe quelle vidéo en extrayant les images qui la composent grâce à la fonction suivante :

```
import cv2
import os
def video_to_frames(video, path_output_dir):
    vidcap = cv2.VideoCapture(video)
    count = 0
    while vidcap.isOpened():
        success, image = vidcap.read()
        if success:
            count+=1
            cv2.imwrite(os.path.join(path_output_dir, '%d.png') % count, image)
        else:
            break
    cv2.destroyAllWindows()
    vidcap.release()
```

- Amélioration 1 :** Dans cette question nous allons « contourner » le problème du cas où le rang de A est strictement inférieur à 2. Pour cela fixons un $\lambda > 0$ et considérons le problème de minimisation suivant :

$$\arg \min_v \frac{1}{2} \|Av - b\|_2^2 + \frac{\lambda}{2} \|v\|_2^2$$

- En vous inspirant du calcul effectué avec la dérivée directionnelle pour les moindres carrés, déterminer l'équation normale (=équation des points critiques) du problème précédent.
- Montrer que cette équation normale admet toujours une solution.
- Modifier alors les algorithmes précédents afin d'ajouter (ou non) ce paramètre λ .

(d) En faisant varier λ commenter l'influence de celui-ci sur les résultats obtenus.

7. **Amélioration 2** : Les systèmes précédents mettent sur le plan « informationnel » à égalité tous les points des voisinages V_h des points (i, j) . Or il est juste d'estimer que le point (i, j) et son voisinage le plus proche comptent plus qu'un voisinage plus éloigné. On définit alors une matrice carrée diagonale W de poids i.e. :

$$\sum_k W_{k,k} = 1$$

et on considère l'équation normale « à poids » suivante :

$$A^T W A v = A^T W b$$

- (a) Modifier les fonctions précédentes pour prendre (ou non) en compte cette matrice W de poids dans le calcul du flux optique. Vous pouvez tester votre algorithme en choisissant comme poids :

$$W_{k,k} = p_k$$

où $p_k := P(X = k)$ où X est une variable aléatoire discrète suivant une loi binomiale.

- (b) En faisant varier le paramètre p de la loi binomiale commenter l'influence de celui-ci sur les résultats obtenus. On tracera dans chaque cas la densité discrète de la loi binomiale choisie et on commentera les résultats obtenus en fonction.

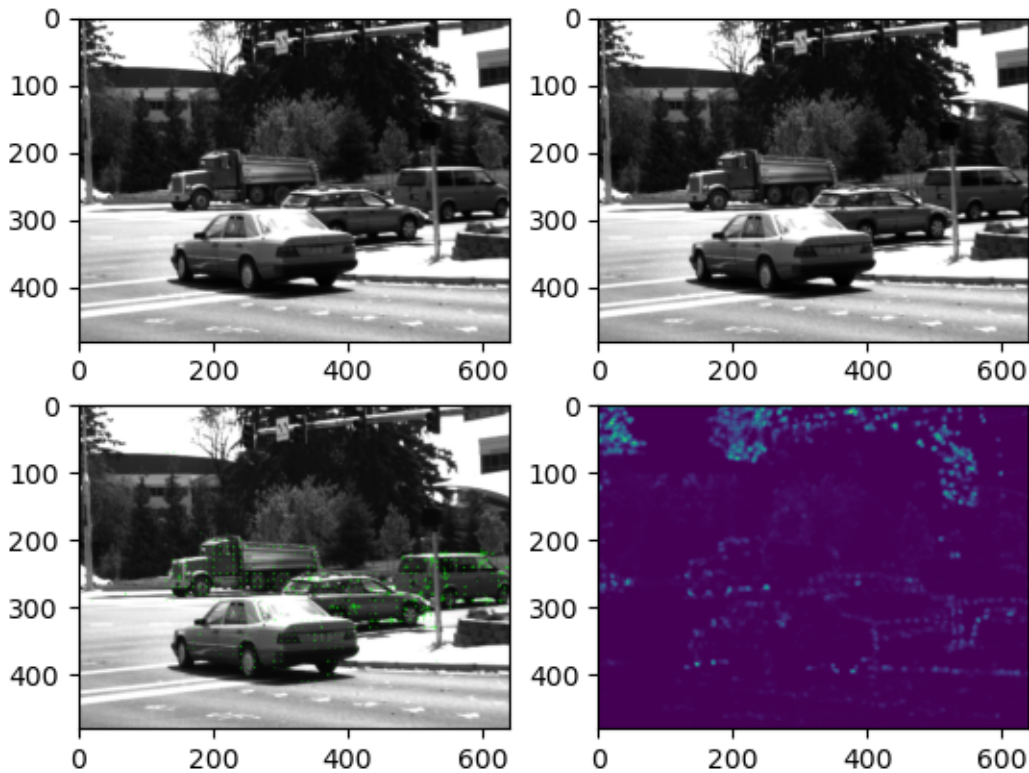


FIGURE 5 – Résolution du flux optique par `MC_flux_calc` pour l'exemple des frames 10 et 11 de Dumptruck. En haut, de gauche à droite, les images 10 et 11. En bas : à gauche, le flux optique et à droite la matrice Λ « l'indicateur » de confiance. Ici nous avons pris $h = 3$.

Troisième partie

En utilisant l'analyse variationnelle.

Revenons sur le problème de la nécessité d'avoir des hypothèses supplémentaires afin de déterminer de manière unique le flux optique. Dans cette partie nous allons supposer que l'on suit le mouvement d'un objet (un objet vu ici comme un ensemble de points matériels) non déformable et par suite que **les variations du flux optique de points proches sur l'objet sont également « proches »**.

Formellement si l'on considère les composantes du flux optique comme dépendantes de leurs positions x, y i.e. :

$$(v_1(x, y), v_2(x, y))$$

on aimerait que :

$$\|\nabla v_1(x, y)\|_2 \quad \text{et} \quad \|\nabla v_2(x, y)\|_2$$

soient les plus petites possibles, la norme du gradient mesurant les variations de la fonction considérée. En fixant un $t \in [0, t_f]$ et $\lambda \in \mathbb{R}_+^*$, le problème de la détermination du flux optique revient alors à déterminer les points critiques de la fonction suivante (on verra l'an prochain que c'est un cas particulier d'**action lagrangienne**) :

$$S^{(t)}(v_1, v_2) := \iint_{[0,1]^2} \left(\left(\frac{\partial f}{\partial t}(t, x, y) + \nabla_{x,y} f(t, x, y)^T \begin{pmatrix} v_1(x, y) \\ v_2(x, y) \end{pmatrix} \right)^2 + \lambda \left(\|\nabla v_1(x, y)\|_2^2 + \|\nabla v_2(x, y)\|_2^2 \right) \right) dx dy$$

On notera $\Omega := [0, 1]^2$ et $\partial\Omega$ notera les bords du carré unité. On pose afin d'alléger les notations :

$$L(v_1, v_2, \partial_x v_1, \partial_x v_2, \partial_y v_1, \partial_y v_2, x, y) := \left(\frac{\partial f}{\partial t}(t, x, y) + \nabla_{x,y} f(t, x, y)^T \begin{pmatrix} v_1(x, y) \\ v_2(x, y) \end{pmatrix} \right)^2 + \lambda \left(\|\nabla v_1(x, y)\|_2^2 + \|\nabla v_2(x, y)\|_2^2 \right)$$

où les $\partial_z u$ notent les dérivées partielles : $\partial_z u := \frac{\partial u}{\partial z}$.

1. Dans cette question nous allons calculer les points critiques de $S^{(t)}$ i.e. les fonctions v_1 et v_2 telles que :

$$D_{h_1} S(v_1, v_2) := \lim_{\epsilon \rightarrow 0} \frac{S^{(t)}(v_1 + \epsilon h_1, v_2) - S^{(t)}(v_1, v_2)}{\epsilon} = 0 \quad \text{et} \quad D_{h_2} S(v_1, v_2) := \lim_{\epsilon \rightarrow 0} \frac{S^{(t)}(v_1, v_2 + \epsilon h_2) - S^{(t)}(v_1, v_2)}{\epsilon} = 0$$

pour toutes directions $h_1 : \Omega \rightarrow \mathbb{R}$ et $h_2 : \Omega \rightarrow \mathbb{R}$ que l'on supposera de classe $\mathcal{C}^\infty$.

(a) Commençons par déterminer $D_{h_1} S(v_1, v_2)$. Fixons $h_1 : \Omega \rightarrow \mathbb{R}$ de classe $\mathcal{C}^\infty$. Démontrer que la dérivée directionnelle partielle de l'action $S^{(t)}$ dans la direction h_1 s'écrit sous la forme :

$$D_{h_1} S(v_1, v_2) = \int \int_{\Omega} \left(\frac{\partial L}{\partial v_1} h_1 + \frac{\partial L}{\partial_x v_1} \partial_x h_1 + \frac{\partial L}{\partial_y v_1} \partial_y h_1 \right) dx dy$$

On pourra utiliser un développement de Taylor à l'ordre 1 de L .

(b) Notons la forme différentielle :

$$dw := -P(x, y) dx + Q(x, y) dy$$

où :

$$\begin{cases} P(x, y) := h_1(x, y) \frac{\partial L}{\partial_y v_1}(v_1, v_2, \partial_x v_1, \partial_x v_2, \partial_y v_1, \partial_y v_2, x, y) \\ Q(x, y) := h_1(x, y) \frac{\partial L}{\partial_x v_1}(v_1, v_2, \partial_x v_1, \partial_x v_2, \partial_y v_1, \partial_y v_2, x, y) \end{cases}$$

En utilisant les notations précédentes rappeler la formule de Green-Riemann associée à l'intégration de la forme différentielle $d\omega$ sur $\partial\Omega$. En déduire une formule d'intégration par parties à deux variables. On rappelle que la notation précédente implique que l'expression $\frac{\partial L}{\partial_y v_1}(v_1, v_2, \partial_x v_1, \partial_x v_2, \partial_y v_1, \partial_y v_2, x, y)$ est vue comme l'évaluation en (x, y) d'une fonction ne dépendant que de x et y .

- (c) Un petit lemme. Afin de souffler un peu, démontrer le résultat simple mais utile suivant. Soit $f : [a, b] \rightarrow \mathbb{R}$ une fonction réelle continue telle que pour toute fonction g continue sur $[a, b]$:

$$\int_a^b f(t)g(t)dt = 0$$

Démontrer que nécessairement f est la fonction nulle. On appelle ce résultat le **lemme fondamental du calcul variationnel**.

- (d) On suppose que $h_1(x, y) = 0$ pour tout $(x, y) \in \partial\Omega$. Dédurre des questions précédentes et du lemme fondamental du calcul variationnel que les points critiques de la dérivée partielle en v_1 de $S^{(t)}$ vérifient sur Ω :

$$\frac{\partial L}{\partial v_1} = \frac{d}{dx} \left(\frac{\partial L}{\partial_x v_1} \right) + \frac{d}{dy} \left(\frac{\partial L}{\partial_y v_1} \right)$$

On obtient ce que l'on appelle **une équation d'Euler-Lagrange** à deux variables. En explicitant L , montrer que dans notre cas cela revient à l'équation suivante :

$$\frac{\partial f^2}{\partial x} v_1 + \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} v_2 = \lambda \Delta v_1 - \frac{\partial f}{\partial x} \frac{\partial f}{\partial t}$$

où $\Delta u(x, y) := \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$, est l'opérateur laplacien.

- (e) En faisant de même pour la dérivée partielle en v_2 de $S^{(t)}$ montrer que l'on obtient le système différentiel suivant vérifié par les fonctions v_1 et v_2 lorsqu'elles sont points critiques de cette action :

$$\begin{cases} \frac{\partial f^2}{\partial x} v_1 + \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} v_2 = \lambda \Delta v_1 - \frac{\partial f}{\partial x} \frac{\partial f}{\partial t} \\ \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} v_1 + \frac{\partial f^2}{\partial y} v_2 = \lambda \Delta v_2 - \frac{\partial f}{\partial y} \frac{\partial f}{\partial t} \end{cases} \quad (2)$$

2. Dans les questions suivantes nous allons résoudre de manière approchée numériquement l'équation (2) précédente. Revenons donc à notre version originelle discrète. Supposons fixées deux images $F^{(r)}$ et $F^{(r+1)}$. La valeur de $f(t_r, x(t_r), y(t_r))$ correspond alors au coefficient :

$$F_{i,j}^{(r)}$$

pour un i et un j dans $\llbracket 1, m \rrbracket$ et $\llbracket 1, n \rrbracket$ respectivement. Avec ces notations, on cherche donc à déterminer le vecteur : $(v_{1,i,j}, v_{2,i,j})^T$. On rappelle (cf. partie 1) que comme pour les matrices un « pas » ne peut pas être plus petit que 1 (=l'écart entre deux pixels) on impose :

$$\Delta x = \Delta y = \Delta t = 1$$

- (a) Démontrer à l'aide de développements de Taylor de f que :

$$\begin{cases} \Delta v_{1,i,j} \simeq v_{1,i+1,j} + v_{1,i-1,j} + v_{1,i,j+1} + v_{1,i,j-1} - 4v_{1,i,j} \\ \Delta v_{2,i,j} \simeq v_{2,i+1,j} + v_{2,i-1,j} + v_{2,i,j+1} + v_{2,i,j-1} - 4v_{2,i,j} \end{cases}$$

On prendra sur les bords que le flux optique est nul.

- (b) En notant :

$$\begin{cases} \bar{v}_{1,i,j} := \frac{1}{4} (v_{1,i+1,j} + v_{1,i-1,j} + v_{1,i,j+1} + v_{1,i,j-1}) \\ \bar{v}_{2,i,j} := \frac{1}{4} (v_{2,i+1,j} + v_{2,i-1,j} + v_{2,i,j+1} + v_{2,i,j-1}) \end{cases}$$

Montrer que l'équation (2) s'écrit sous forme discrète matricielle :

$$(\bar{\lambda} I_2 + b b^T) v = \bar{\lambda} \bar{v} - \alpha b$$

où $\bar{\lambda} \in \mathbb{R}_*^+$, $v := (v_{1,i,j}, v_{2,i,j})^T$, $\bar{v} := (\bar{v}_{1,i,j}, \bar{v}_{2,i,j})^T$ et b un vecteur à déterminer et α un scalaire à déterminer. Démontrer alors que ce système est inversible et en déduire l'expression suivante matricielle :

$$v = \bar{v} - \left(\frac{b^T \bar{v} + \alpha}{\bar{\lambda} + \|b\|_2^2} \right) b$$

Cette expression permet de faire apparaître une équation de point fixe de la forme :

$$v = g(v)$$

où : $g(v) := \bar{v} - \left(\frac{b^T \bar{v} + \alpha}{\bar{\lambda} + \|b\|_2^2} \right) b$. On pose alors en reprenant la démarche effectuée en cours la méthode itérative suivante :

$$\begin{cases} V_1^{(0)} \text{ et } V_2^{(0)} \\ v^{(k+1)} = \overline{v^{(k)}} - \left(\frac{b^T \overline{v^{(k)}} + \alpha}{\bar{\lambda} + \|b\|_2^2} \right) b \quad k \geq 1 \end{cases}$$

- (c) En utilisant les questions précédentes définissez la fonction **var_flux_calc** qui prendra en entrée deux images consécutives d'une vidéo $F^{(r)}$ et $F^{(r+1)}$ et qui renverra deux matrices V_1 et V_2 approximations du flux optique. Comme pour les autres méthodes itératives vues en classe, on fixera une tolérance (=erreur) ϵ ainsi qu'un nombre d'itération maximal. L'algorithme à programmer est donc le suivant :

Algorithme 2 Résumé de l'algorithme de calcul du flux optique par l'approche variationnelle

Fixer $\lambda > 0$, $\epsilon > 0$ **et** $nbitermax > 0$.

Calculer les $\partial_x F_{i,j}^{(r)}$, $\partial_y F_{i,j}^{(r)}$ **et** $\partial_t F_{i,j}^{(r)}$ **pour tout** $(i, j) \in \llbracket 1, m \rrbracket \times \llbracket 1, n \rrbracket$.

Initialiser les matrices de taille (m, n) des composantes du flux optique : $V_1^{(0)}$ **et** $V_2^{(0)}$.

Tant que l'erreur est plus grande que $\epsilon > 0$ **et** $k < nbitermax$:

Calculer pour chaque pixel i, j :

$$v^{(k+1)} = \overline{v^{(k)}} - \left(\frac{b^T \overline{v^{(k)}} + \alpha}{\bar{\lambda} + \|b\|_2^2} \right) b$$

Vous pourrez tester votre algorithme sur les données fournies sur Moodle et afficher le résultat obtenu en vous inspirant du code suivant (on rappelle que dans OpenCV les conventions lignes et colonnes sont inversées par rapport aux conventions mathématiques usuelles) :

```
image=np.copy(img1)
image=cv2.cvtColor(image,cv2.COLOR_GRAY2RGB)
for i in range(0,m,10):
    for j in range(0,n,10):
        u,v=int(V1[i,j]),int(V2[i,j])
        if np.sqrt(u**2+v**2)>1:
            image = cv2.arrowedLine(image, (j,i), (j+v,i+u), (0,255,0),1, tipLength = 0.5)
cv2.imshow("affichage_final", image)
```

- (d) En faisant varier λ commenter l'influence de celui-ci sur les résultats obtenus.
- (e) En utilisant la librairie OpenCV et `cv2.VideoWriter` (cf. fascicule de cours), réaliser une sortie vidéo des calculs du flux optique sur une série d'images. Si les images fournis en exemple ne vous suffisent pas vous pouvez tester votre algorithme sur n'importe quelle vidéo en extrayant les images qui la composent grâce à la fonction donnée en partie précédente.
3. Dans l'article original, dont s'inspire cette partie du BE, de B.K.P. Horn et B.G. Schunck, « Determining optical flow » dans Artificial Intelligence, vol 17, pp 185–203, 1981, les formules d'approximations sont plus complexes. Les auteurs choisissent les formules suivantes :

$$\begin{cases} \frac{\partial f}{\partial x}(t_r, x(t_r), y(t_r)) \simeq \frac{1}{4} \left(F_{i+1,j}^{(r)} - F_{i,j}^{(r)} + F_{i+1,j+1}^{(r)} - F_{i,j+1}^{(r)} + F_{i+1,j}^{(r+1)} - F_{i,j}^{(r+1)} + F_{i+1,j+1}^{(r+1)} - F_{i,j+1}^{(r+1)} \right) \\ \frac{\partial f}{\partial y}(t_r, x(t_r), y(t_r)) \simeq \frac{1}{4} \left(F_{i,j+1}^{(r)} - F_{i,j}^{(r)} + F_{i+1,j+1}^{(r)} - F_{i+1,j}^{(r)} + F_{i,j+1}^{(r+1)} - F_{i,j}^{(r+1)} + F_{i+1,j+1}^{(r+1)} - F_{i+1,j}^{(r+1)} \right) \\ \frac{\partial f}{\partial t}(t_r, x(t_r), y(t_r)) \simeq \frac{1}{4} \left(F_{i,j}^{(r+1)} - F_{i,j}^{(r)} + F_{i+1,j}^{(r+1)} - F_{i+1,j}^{(r)} + F_{i,j+1}^{(r+1)} - F_{i,j+1}^{(r)} + F_{i+1,j+1}^{(r+1)} - F_{i+1,j+1}^{(r)} \right) \end{cases}$$

et pour le laplacien :

$$\begin{cases} \Delta v_{1,i,j} \simeq \frac{1}{2} (v_{1,i-1,j} + v_{1,i,j+1} + v_{1,i+1,j} + v_{1,i,j-1}) + \frac{1}{4} (v_{1,i-1,j-1} + v_{1,i-1,j+1} + v_{1,i+1,j+1} + v_{1,i+1,j-1}) - 3v_{1,i,j} \\ \Delta v_{2,i,j} \simeq \frac{1}{2} (v_{2,i-1,j} + v_{2,i,j+1} + v_{2,i+1,j} + v_{2,i,j-1}) + \frac{1}{4} (v_{2,i-1,j-1} + v_{2,i-1,j+1} + v_{2,i+1,j+1} + v_{2,i+1,j-1}) - 3v_{2,i,j} \end{cases}$$

Créer une fonction **var_flux_calc_v2** qui utilise les formules précédentes d'approximations. Comparer les résultats obtenus avec ceux de la question précédente.

Quatrième partie

Comparaisons et applications.

Cette partie est une synthèse des parties précédentes.

1. Comparer sur des mêmes paires (ou plus) d'images les résultats des algorithmes de la partie 3 et 4. Vous commenterez les différences et les ressemblances des résultats obtenus.
2. **Segmentation d'image.** L'obtention du flux optique permet de réaliser des segmentations d'images et par suite d'une séquence vidéo. Segmenter une image consiste à regrouper entre elles des parties (=des pixels) d'une image en fonction de critères. Nous allons voir une version simplifiée de réalisation d'une segmentation (il faudrait développer un peu plus les calculs pour avoir une version plus robuste). Ici nous allons utiliser pour tout point (i, j) d'une image la norme du vecteur : $v^{(i,j)} := (v_{1,i,j}, v_{2,i,j})^T$ du flux optique calculé.
 - (a) Créer une matrice S de taille (m, n) contenant pour chaque (i, j) la norme du vecteur $v^{(i,j)}$.
 - (b) En utilisant la matrice précédente et en fixant un réel incarnant un seuil $s \in \mathbb{R}_*^+$ créer une matrice $M \in \mathcal{M}_{m,n}(\mathbb{R})$ que l'on appellera **masque** de l'image telle que :

$$M_{ij} = \begin{cases} 1 & \text{si } \|v^{(i,j)}\|_2 \geq s \\ 0 & \text{sinon} \end{cases}$$

On pourra s'aider de la valeur moyenne de S (`np.mean(S)` dans Python) afin de choisir des valeurs de s « convenables ».

- (c) Afficher sur des exemples d'images $F^{(r)}$ les images obtenues en faisant le produit d'Hadamard (=produit termes à termes) avec la matrice M de masque et la matrice complémentaire i.e :

$$F_1 := F^{(r)} \odot M \quad \text{et} \quad F_2 := F^{(r)} \odot (1_{m,n} - M)$$

où $\odot$ note le produit d'Hadamard et $1_{m,n}$ note la matrice de taille (m, n) ne comportant que des 1.



FIGURE 6 – Un exemple de segmentation rapide produit avec des images du dossier Basketball. Ici les paramètres n'ont pas été optimisés et le calcul a été effectué avec l'algorithme de la partie 3. On voit par exemple que le déplacement de l'ombre du ballon est également en partie détecté.

- (d) Réaliser une vidéo à partir d'images segmentées.
3. **Ouverture : Tracking.**
 - (a) En se fixant un point (x_0, y_0) sur la première image $F^{(0)}$ et en calculant le flux optique de ce point sur les images successives créer une vidéo où le flux optique issu de (x_0, y_0) est représenté sur chaque image de la vidéo.
 - (b) Variation : De même en se fixant un point (x_0, y_0) sur la première image $F^{(0)}$ et en calculant le flux optique de ce point sur les images successives créer une vidéo où le flux optique issu de (x_0, y_0) est toujours au centre de la vidéo. On ajoutera des pixels nuls aux endroits où l'image n'est pas définie.